LAPORAN PRAKTIKUM 3
ENKAPSULASI DAN KONSTRUKTOR



Disusun Oleh:

KHOHAR MUHAMAD FATAHURROHMAN

254107020021

Kelas TI 2B

PROGRAM STUDI TEKNIK INFORMATIKA

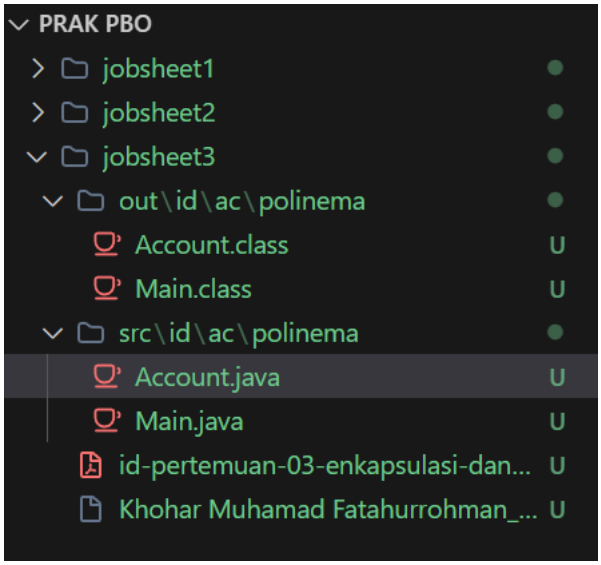
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

September 2026

Langkah Percobaan

a. Struktur Project



Penjelasan Struktur Project:

Proyek ini bernama bank-mini (atau jobsheet3) dan diorganisasikan menggunakan struktur package id.ac.polinema. Package ini berfungsi untuk mengelompokkan kelas-kelas Java yang berkaitan, menghindari konflik nama kelas (name collision), serta mempermudah manajemen access modifier. Di dalam package ini terdapat dua file utama:

1.Account.java: Bertindak sebagai blueprint atau domain model yang merepresentasikan entitas rekening bank. Kelas ini menyimpan data (state) dan perilaku (behavior) yang dimiliki sebuah akun.

2. Main.java: Bertindak sebagai entry point (titik awal jalannya program). File ini berisi method main yang digunakan untuk menginstansiasi objek Account dan menguji jalannya alur logika program.

b. Implementasi & Menjalankan Program

Account.java U jobsheet3\src\id\ac\polinema\Account.java\ Account

```java
package id.ac.polinema;

public class Account {
    private String accountNumber;
    private String ownerName;
    private double balance;
    private double dailyWithdrawalLimit;

    public Account(String accountNumber, String ownerName, double balance, double dailyWithdrawalLimit) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
        this.dailyWithdrawalLimit = dailyWithdrawalLimit;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public double getBalance() {
        return balance;
    }

    public boolean deposit(double amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }

    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance || amount > dailyWithdrawalLimit) {
            return false;
        }
        balance -= amount;
        return true;
    }

    public void printInfo() {
        System.out.println(accountNumber + " - " + ownerName + " - balance: " + balance);
    }
}
```

Main.java U jobsheet3\src\id\ac\polinema\Main.java

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Account limited = new Account(accountNumber: "A005", ownerName: "Fajar", balance: 1000000, dailyWithdr… 200000);
        System.out.println("Withdraw 300000 allowed? " + limited.withdraw(amount: 300000));
        limited.withdraw(amount: 150000);
        limited.printInfo();
    }
}
```

```
Problems  Debug Console  Output  Terminal  Ports

PS D:\KULIAH\SEMESTER 3\PRAK PBO> cd jobsheet3
PS D:\KULIAH\SEMESTER 3\PRAK PBO\jobsheet3> javac -d out src/id/ac/polinema/*.java
PS D:\KULIAH\SEMESTER 3\PRAK PBO\jobsheet3> java -cp out id.ac.polinema.Main
Withdraw 300000 allowed? false
A005 - Fajar - balance: 850000.0
PS D:\KULIAH\SEMESTER 3\PRAK PBO\jobsheet3>
```

Penjelasan Kode, Konsep OOP, dan Kendala:

Pada kode di atas, kita menerapkan prinsip Encapsulation dan Information Hiding. Semua atribut pada kelas Account (seperti accountNumber, ownerName, balance, dan dailyWithdrawalLimit) dideklarasikan dengan tipe akses private. Ini dilakukan untuk mencegah kelas lain secara langsung mengakses dan mengubah nilai variabel tersebut seenaknya.

Sebagai gantinya, untuk memberikan nilai awal, program diwajibkan melewati sebuah Constructor dengan argumen lengkap Account(String accountNumber, String ownerName, double balance, double dailyWithdrawalLimit). Sedangkan untuk mengambil nilai atribut (tanpa hak untuk mengubahnya bebas), digunakan method Getter (seperti getAccountNumber()).

Perubahan state / nilai atribut hanya bisa dilakukan melalui method yang terkontrol yaitu deposit() dan withdraw(). Kedua method tersebut telah dilengkapi dengan Validasi (contoh: mengecek jika nilai penarikan melebihi saldo atau limit harian). Method tersebut juga mengembalikan nilai boolean untuk memberitahu Main apakah transaksinya sukses atau ditolak.

Kendala: Pada awalnya sempat muncul error constructor cannot be applied. Solusi: Saya memperbaiki argumen pada pemanggilan new Account(...) di Main.java agar urutan tipe datanya (String, String, double, double) sesuai dengan yang diminta pada parameter konstruktor.

Hasil & Pembahasan

Berdasarkan hasil akhir dari program ini, penerapan Enkapsulasi berjalan dengan sangat baik:

- Access Modifier: Penggunaan modifier private pada variabel kelas berhasil menyembunyikan implementasi dan data internal dari kelas eksternal. Interaksi dari luar hanya dibuka melalui method public.
- Getter/Setter: Atribut pada Account (misalnya accountNumber) didesain menjadi read-only dengan hanya menyediakan method Getter tanpa method Setter. Hal ini sesuai dengan skenario dunia nyata di mana sebuah Nomor Rekening yang sudah dibuat tidak boleh diubah-ubah nilainya.
- Konstruktor Custom: Saya telah menerapkan satu konstruktor parameter (custom constructor) yang memaksa agar saat sebuah objek dibuat (new

Account(...)), data-data essential seperti nomor rekening, nama pengguna, saldo awal, dan limit penarikan harian harus sudah disediakan.

- Konsep Enkapsulasi ini menjadikan kelas lebih aman (safe state) karena segala logika pemotongan uang dan cek limit terpusat pada method withdraw() (tersentralisasi).

Jawaban Tugas Mandiri:

1. Mengapa mengembalikan nilai boolean dari deposit()/withdraw() lebih aman?

   Jawaban: Karena dengan nilai kembalian boolean, caller (pemanggil method) tahu pasti status kegagalan atau keberhasilan transaksi tanpa harus mengecek saldo kembali secara manual. Hal ini memungkinkan program utama menangani status kegagalan dengan menampilkan notifikasi yang tepat.

2. Atribut mana yang read-only?

Jawaban: Atribut accountNumber wajib dibuat read-only agar tidak ada modifikasi secara ilegal atau tidak sengaja setelah akun pertama kali dibuat di instansiasi konstruktor.

Kesimpulan

Dari praktikum Jobsheet 3 ini, dapat disimpulkan bahwa prinsip Enkapsulasi sangat penting dalam Pemrograman Berorientasi Objek untuk melindungi data dari perubahan yang tidak terencana. Dengan kombinasi penggunaan modifier private pada atribut, pembuatan custom Constructor untuk inisialisasi awal, penerapan Getter untuk read-only access, serta sentralisasi logika validasi (business rules) pada method yang bersangkutan, kita dapat menciptakan class yang kokoh, mudah di-maintenance, dan aman dari bug (seperti saldo negatif atau perubahan nomor rekening tanpa izin).

Lampiran

Link Repositori GitHub untuk Source Code Lengkap:

https://github.com/helfa25/PRAKTIKUM-PBO.git